

Detection and Mitigation of Volumetric DDoS Attacks using Adaptive Rate-Limiting in P4-DPDK

Samia Choueiri, Ali Mazloun, Sergio Elizalde, Elie F. Kfoury, Jorge Crichigno
University of South Carolina, USA
{choueiri, amazloun, elizaldis, ekfoury}@email.sc.edu, jcrichigno@cec.sc.edu

Abstract—Distributed Denial of Service (DDoS) attacks are increasingly targeting network environments. This paper presents a high-performance, adaptive system for detecting and mitigating volumetric DDoS attacks using P4-DPDK. The proposed system operates entirely in the user space, leveraging multicore CPUs and SmartNICs to process traffic at line rate while enabling flexible and efficient control plane operations. DDoS detection is implemented in a linear prediction model that forecasts traffic based on historical observations and dynamically adjusts rate-limiting thresholds. The hyperparameters for the model are tuned using an optimization algorithm. The system is evaluated on the FABRIC testbed and tested using real traffic traces. Experimental results demonstrate robust mitigation against diverse attack types, effective adaptation to real-world traffic, and reduced packet loss under high-throughput conditions approaching 100 Gbps, compared to Suricata-DPDK implementations.

Index Terms—DDoS, DPDK, P4, PNA, SmartNIC, Linear Prediction.

I. INTRODUCTION

Network environments are progressively being targeted by large-scale Distributed Denial of Service (DDoS) attacks [1]. These attacks are conducted by malicious individuals who manage a network of compromised hosts. In recent years, there has been a significant increase in both the size and complexity of DDoS attacks, with traffic volume having doubled over the last year [1].

Current methods to mitigate these attacks fall into software and hardware-based categories. Software solutions are highly flexible and can be easily implemented across different networks, but they also encounter difficulties in managing high traffic volumes, particularly during DDoS events [2]. These limitations arise because the standard network stack is used, which results in significant latency due to the overhead associated with packet processing. Software solutions include Artificial Intelligence (AI) and Machine Learning (ML) techniques, which can analyze traffic patterns in real-time. However, these approaches often require training using specialized hardware and substantial computational resources, making them less practical for high-speed deployments [3].

On the other hand, hardware-based solutions like P4-programmable data plane switches or Field-Programmable Gate Arrays (FPGAs) offer speed advantages over software methods [4], [5]. However, they have several drawbacks; 1) They lack the flexibility necessary for complex packet processing logic. 2) They demand specialized skills for programming, operation, and maintenance, complicating their integration into networks largely dominated by fixed-function devices. 3)

Their large-scale deployment can be costly, and they are often unsupported in cloud environments, which is problematic as many organizations have moved to the cloud.

A fair approach is to leverage the flexibility and simplicity of deploying software solutions with acceleration to avoid significantly compromising performance. The Data Plane Development Kit (DPDK) strikes this balance by bypassing the kernel and eliminating the packet processing overhead of the traditional network stack. It efficiently uses hardware to distribute and balance packet loads across CPU cores. Moreover, DPDK is P4-programmable and compatible with most modern Network Interface Cards (NICs), providing an accessible option for high-performance packet processing.

This paper presents a high-performance, adaptive system for detecting and mitigating a wide range of volumetric DDoS attacks using P4-DPDK. The system operates entirely in the user space, leveraging multicore CPUs and SmartNICs to enable flexible and high-speed packet processing. It employs a lightweight linear prediction model to forecast traffic trends based on historical observations and dynamically adjusts rate-limiting thresholds to detect floods without relying on hard-coded values. The DDoS detection and mitigation logic is implemented in P4 and compiled into a DPDK packet processing pipeline using the p4c-dpdk compiler. The system is evaluated on the FABRIC testbed using both synthetic and real-world traces. The results demonstrate high adaptability and reduced packet loss under attack. The source code of the system is publicly available [6].

The contributions of the paper are summarized as follows:

- Designing a predictive DDoS mitigation system based on a linear prediction model integrated into a P4-DPDK pipeline.
- Optimizing the detection sensitivity through hyperparameter tuning of the linear model, ensuring accurate flood detection while minimizing false alerts.
- Evaluating the system's adaptability to real-world network traces using datasets from a backbone network, and testing its scalability on the FABRIC testbed.
- Demonstrating superior performance compared to Suricata-DPDK in terms of packet loss and scalability across increasing CPU core counts.

II. RELATED WORK

A. DDoS Defenses over P4 Programmable Data Planes

DDoS protection schemes that use P4 programmable data planes operate at the line rate [7]. Due to P4's adaptability, detection and mitigation strategies can be adjusted in response to innovative attack strategies. Current P4-based solutions employ probabilistic data structures to monitor flow statistics at scale (e.g., MV-Sketch [8], POSEIDON [9], and Jaqen [10]). The primary drawback of these solutions is their demand for infrastructure upgrades, which adds cost and complexity. Additionally, such systems are not suitable for cloud environments where resources are dynamically allocated.

B. DDoS Defenses over DPDK

DPDK effectively addresses the deployment challenges associated with P4 programmable data planes, at the cost of having relatively lower performance. DPDK's performance can reach 200Gbps per NIC, and can be scaled by adding more NICs and CPU cores. Today, most high-end NICs come equipped with DPDK support. The capabilities of DPDK have been investigated in various DDoS applications [11]–[14], demonstrating notable performance improvements. Despite these advancements, there are limitations that hinder scalability and flexibility. C lacks the domain-specific abstractions required for clearly expressing packet parsing and match-action logic, leading to complex, error-prone implementations that are difficult to maintain. Adding new protocols or modifying behavior in C often requires extensive rewrites, whereas P4's high-level, declarative model allows rapid iteration and easier integration with control planes. Moreover, P4 reduces the likelihood of memory-related bugs, improves code portability across hardware targets, and is better suited for formal testing and verification.

III. BACKGROUND

A. P4 Language and Portable NIC Architecture (PNA)

Networking devices known as programmable data planes offer customizable packet processing functions. P4, which stands for Programming Protocol-independent Packet Processor, is a domain-specific language designed for programming these devices [15]. The Portable NIC Architecture (PNA) provides the P4 language with structures and essential functionalities tailored for SmartNICs. It has three main programmable components: the parser, the main control, and the deparser. The parser extracts the headers from the incoming packets. The main control block processes these extracted headers along with any intermediate computations. Lastly, the deparser reassembles the packets and prepares them for transmission.

B. Data Plane Development Kit (DPDK)

DPDK is a software acceleration framework optimized for packet processing. It uses kernel bypass to enable direct interaction between the NIC and the DPDK Poll Mode Driver (PMD) in the user space. PMD eliminates the per-packet kernel interrupts by polling the incoming packets in batches. This

packet processing acceleration technique reduces the overhead that is usually caused by the kernel's networking infrastructure in traditional approaches [16]. DPDK is conventionally developed in C, which requires developers to write detailed and efficient code to set up packet processing pipelines. Although this method provides significant performance and adaptability, it requires a strong knowledge of low-level networking and the DPDK API.

C. P4-DPDK

P4-DPDK acts as a link between P4 and DPDK, enabling developers to write a P4 code that is subsequently converted into a DPDK pipeline. This integration merges P4's expressiveness with DPDK's performance advantages. The p4c compiler [17] uses a DPDK backend known as p4c-dpdk [18], and converts the P4 code into a specification file (.spec) that is needed to program the DPDK Software Switch (SWX) pipeline [19]. The DPDK libraries are used to build the packet processing pipeline and generate the files to run the application. P4 and DPDK have been increasingly adopted for cybersecurity applications such as DDoS detection and mitigation, deep packet inspection, traffic analysis, and domain name inspection [20]–[22].

D. Linear Prediction

A linear system is one in which the output is generated as a linear combination of its current and past inputs, as well as its previous outputs. This means that the system follows the principles of superposition and scaling, ensuring that its behavior can be described using linear equations. Linear prediction is a widely used technique in time series analysis. It relies on linear prediction coefficients, which describe how past values in a time series can be used to estimate future values [24]. The structure of the linear Prediction Model is described in Fig. 1. The system relies on features of training data that are known to fall into a specific category. After this portion of the data is used in the learning algorithm, the parameters of the Linear Prediction Function are tuned. At this stage, the function takes as an input the features of the current observation denoted by x and computes the next predicted value $\hat{x}$.

IV. PROPOSED SYSTEM

Consider Fig. 2 which shows a high-level overview of the proposed system. The system employs a DDoS attack

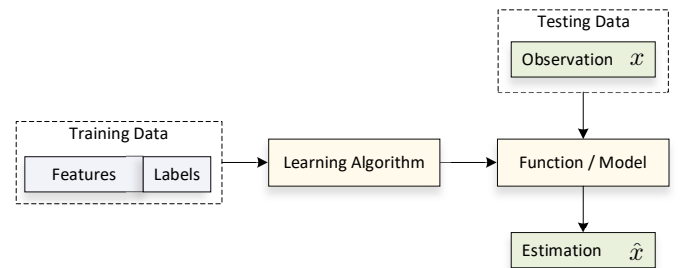


Fig. 1. The structure of a Linear Prediction Model [23].

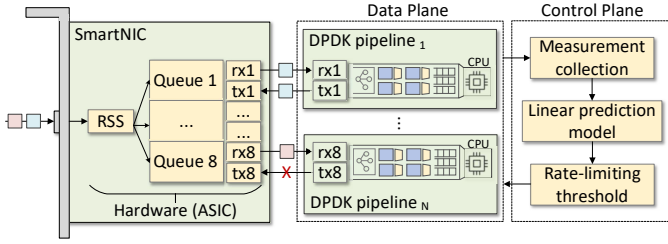


Fig. 2. Proposed system architecture. DDoS mitigation pipelines are deployed on CPU cores. Based on the implemented logic in the pipelines and the calculated rate-limiting threshold, the packets are either allowed or dropped.

detection and mitigation pipeline operating in the user space on multicore CPUs. Using DPDK, packets bypass the kernel entirely. When a packet is received, the SmartNIC directs it to a designated pipeline running on a specific CPU core using the in-hardware Receive Side Scaling (RSS) hashing module [25]. As the packet reaches the pipeline, it undergoes processing by the algorithm in the data plane which also writes the count of packets to registers. Simultaneously, The control plane continuously reads the number of packets received every second and uses these measurements to forecast the number of packets in the next observation. Based on the expected value, the allowed number of packets to be processed is pushed to the data plane which will drop the packets that exceed the allowed range. The implemented code detects and mitigates seven different DDoS attacks: 1) UDP flood, 2) SYN flood, 3) SYN-ACK flood, 4) ACK flood, 5) FIN flood, 6) Elephant Flow, and 7) ICMP flood.

A. Throughput forecasting Using the Linear Prediction Model

The following Variable and constants are considered for the mathematical representation of the model:

- X : Set of observed packets per second.
- k : Number of past samples used for prediction
- α : Linear prediction coefficient

A sequence of the last k observed packets per-second counts is denoted in X :

$$X = [x_1, x_2, \dots, x_k]$$

The aim is to predict the upcoming element of $\hat{x}$ by calculating the average of the differences of the observations in X and summing it with the last observation x_k as follows:

$$\hat{x} = \frac{1}{k-1} \sum_{i=1}^{k-1} (x_{i+1} - x_i) + x_k$$

The linear prediction coefficient α is a constant value ranging between zero and one. Based on the predicted value $\hat{x}$ and α , the allowed number of packets per second x_{allowed} is determined by increasing the predicted amount of packets by a certain offset as shown in the following equation:

$$x_{\text{allowed}} = \hat{x} * (1 + \alpha)$$

The calculated amount of packets allowed x_{allowed} is needed for two reasons: 1) to make sure that the network is able to observe a normal increase in traffic, and 2) to limit the rate of processed packets when a flood is detected. This value is written to a register and pushed to the data plane to make sure that the processed number of packets in the pipeline will not exceed the allowed amount x_{allowed} .

B. DDoS Attack Detection and Mitigation Methods

The system checks for a DDoS attack by calculating the error ratio of the predicted value of $\hat{x}$ to the next observed value x as follows:

$$e = \frac{\hat{x}}{x}$$

The value of e is used to calculate how close the expected value $\hat{x}$ is to the actual value x . Therefore a good prediction will result in a value of e that is close to one which also indicates a normal behavior in the network.

When an attack is initiated, the observed value x will be large compared to the expected value $\hat{x}$ resulting in a sudden drop in the value of e . If $e < (1 - \alpha)$, a flood is detected. At this stage, only the allowed number of packets x_{allowed} is processed by the P4-DPDK pipeline in the data plane and the remaining packets are dropped. The attack is considered to be over when the ratio e recovers and satisfies the inequality $e > (1 - \alpha)$. For this to happen, the observed number of packets per second x has to drop below the allowed number of packets x_{allowed} and the network will recover to its normal behavior, and the linear prediction resumes.

C. Optimization Model of the Linear Prediction Model Hyperparameter

The linear prediction model relies on two parameters that influence the overall accuracy of the system. The number of previously observed samples, denoted by k affects the accuracy of the predicted value $\hat{x}$. Considering more historical data results in a more informed prediction of future observations. However, a higher value of k introduces a trade-off as it delays the point at which the system can begin generating predictions. The linear prediction coefficient α plays an important role in calibrating the system's sensitivity. The model needs to be sensitive enough to trigger alerts every time a flood is detected. An overly sensitive system would observe fluctuations in legitimate traffic as floods, therefore triggering false alerts that lead to increased packet loss. To find the most suitable parameters, an optimization problem is modeled as follows:

$$\begin{aligned} \min_{\alpha, k} \quad & \left(\frac{1}{T} \sum_{t=1}^T |x_t - \hat{x}_t| - \frac{1}{\alpha} + \beta k \right) \\ \text{subject to} \quad & k \geq 3, \quad 0 < \alpha < 1, \quad \forall e < 1 - \alpha \end{aligned}$$

The considered decision variables are α and k . The objective function to be minimized consists of three components that contribute to the overall cost: 1) The mean Absolute Error (MAE). It is calculated by averaging the absolute value of the difference between the observed value x and the predicted

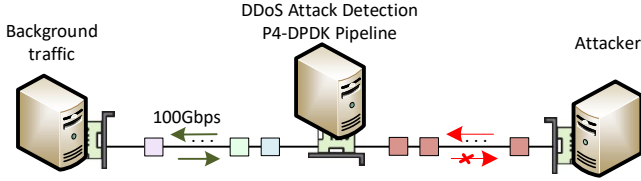


Fig. 3. Experiment topology. The P4-DPDK pipeline detects the flood traffic generated by the attacker, while it processes background traffic at a high rate.

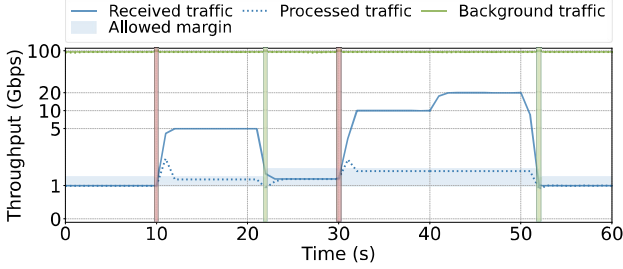


Fig. 4. Detection and mitigation of a flood attack using the linear prediction model. The solid lines refer to the generated traffic, and the dotted lines refer to the processed (reflected) traffic.

value $\hat{x}$ every second t over an interval of time T measures in seconds. This component has to be minimal to ensure that the system produces the best predictions. 2) The smallest value of α ranging from 0 to 1 while the ratio e remains less than $1 - \alpha$. This ensures the system's ability to detect attacks while avoiding any false positive alerts. Finally, 3) The smallest amount of previous observations k being at least equal to three which is the minimal number of observations needed to calculate a prediction. k is multiplied by β , which is a weight that can be calibrated to specify the impact of the value of k on the cost function.

V. IMPLEMENTATION AND EVALUATION

The system is implemented and deployed on FABRIC [26], an international testbed funded by the NSF for large-scale research and experimentation. The FABRIC infrastructure comprises a network of distributed equipment located at different sites. It offers resources including servers, NICs, high-speed links, and more.

The implemented topology, shown in Fig. 3, consists of three nodes, each equipped with an NVIDIA ConnectX-6 NIC (100Gbps). The servers are located in the same FABRIC site. DPDK-pktgen [27] is used to generate both background traffic and attack traffic. The background traffic consists of packets from a range of flows (i.e., unique 5-tuple). The P4-DPDK pipeline reflects all incoming processed packets. This allows measuring the performance of the system by comparing the number of sent packets with the number of received (reflected) packets. The attacker node generates different types of attacks that will be detected and mitigated by the P4-DPDK pipeline.

A. Rate-limiting Based on Linear Prediction

In this experiment, the system continuously monitors network behavior and uses a linear prediction model to calculate the allowed margin of throughput. This model analyzes historical traffic patterns to predict the expected number of packets under normal conditions. An attack is detected by comparing the predicted allowed margin with the most recent observed packet count. For this experiment, a SYN flood attack is evaluated among six other attacks that the system is able to mitigate in a similar way. The throughput prediction is calculated from the throughput history of size k that is equal to 4 in this example. The allowed margin and the detection of the flood are in function of α , which is set to 0.2 in this example. Consider Fig. 4. The system starts by detecting the normal behavior of the system based on the benign traffic received in the first 10 seconds as represented by the blue solid line. All packets were also processed as represented by the blue dotted line. In the tenth second (highlighted in red), a flood is detected. While the received traffic is above the allowed range (shaded blue area), the system only processes a number of packets that matches the allowed range. The attack is claimed to be finished when the flood traffic drops below the allowed range as demonstrated in the twentieth second (highlighted in green). At this stage, the system will resume its normal activity and continue the prediction process and the estimation of the allowed range. Another flood is demonstrated at the thirtieth second and ends at the fiftieth second. The proposed system detects the attacks while it processes legitimate traffic at a high rate approaching 100 Gbps of throughput.

B. Evaluating the Impact of the Linear Prediction Coefficient

The aim of this experiment is to clarify the effect of the Linear Prediction Coefficient α on the function of the system. Fig. 5 describes the behavior of the system when subject to a flood attack initiated at the tenth second with different values of α (0.05, 0.1, 0.25, and 0.5). It is concluded from this experiment, that α is a parameter that specifies the sensitivity of the system to sudden increases in the throughput. For a small value for α such as 0.05, the allowed range is a small offset above the predicted value, and a small rise in the observed throughput is sufficient to claim an attack. Whereas, for a bigger value of α such as 0.5, the allowed range is a remarkable offset above the predicted value, and a more aggressive rise in the observed throughput is needed to claim an attack. Therefore, the smaller the value of α , the higher the sensitivity of the system. It is strongly required that the system be sensitive enough to detect the flood in the system, but a highly sensitive system would generate a great amount of false detections. Ideally, the value of α should be well calibrated based on throughput recorded during the system's normal activity, taking into account the fluctuations in the throughput.

C. Evaluating the Adaptability of the System with Real-traffic

This experiment evaluates the system with open-source real traffic captured from different Measurements and Analyses

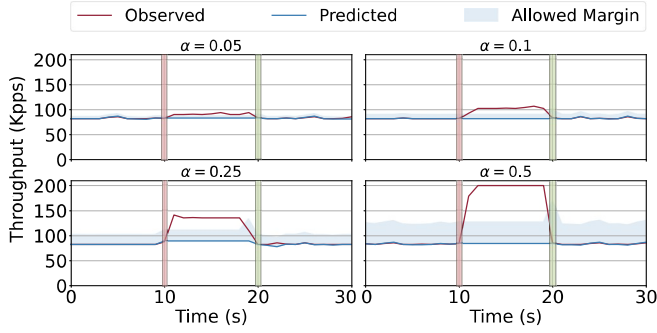


Fig. 5. Behavior of the proposed system with different values of α . A SYN Flood attack is initiated at the tenth second shaded in red and ends the time sample shaded in green.

on the WIDE Internet (MAWI) [28] and Center for Applied Internet Data Analysis (CAIDA) [29] datasets. Additionally, a 30-second SYN flood attack dataset is used to check if the system will correctly detect the floods [30]. In Fig. 6, the three different traces captured from different sources and different days are shown with the resulting predictions and allowed margin from the linear prediction model. Subsequently, the SYN flood flow enters. All flood attacks were correctly identified as attacks (highlighted in red). The required feature is extracted from the traces as they are processed by calculating the observed number of SYN packets per second. The linear prediction model starts by taking the first chunk of data, considering an initial guess of α and k as it learn the optimal value to be used in the next chunk of data. The CAIDA traffic in Fig. 6 (a), is a five-minute trace. The optimization of the hyperparameters is scheduled every 60 seconds. This dataset consists of a slightly higher amount of SYN packets per second on average compared to the remaining two traces. It also has the smallest standard deviation, pointing to the stability of the observations. In Fig. 6 (b) and (c), the MAWI traces consist of network traffic data captured throughout a day as part of the project "A Day in the Life of the Internet (DITL)". The traces are captured as 15-minute time slots at which the hyperparameters adapt to the network operation. The 2024 12-hour trace in Fig. 6 (b) was entirely taken by the linear prediction model as normal traffic. On the other hand, in the 2025 24-hour trace in Fig. 6 (c), more fluctuations are observed as indicated by the standard deviation. Some of these fluctuations spike up for one second. The system has identified several spikes within the benign traffic as potential flood attacks. While these may initially appear to be false positives, we cannot definitively classify them as such. The dataset was captured from a real-world network environment, where the presence of actual flood attacks is highly probable. Therefore, these detections could reflect legitimate threats. The accuracy of the proposed system remains above 99.99%.

D. Comparing Packet-loss in P4-DPDK and Suricata-DPDK

Suricata [31] is a well-known open-source network Intrusion Detection System (IDS) and Intrusion Prevention System

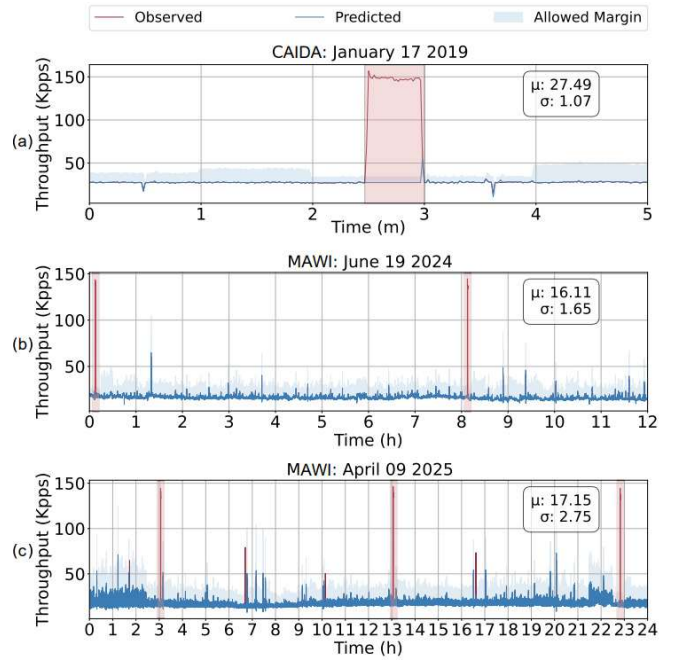


Fig. 6. Evaluation with real traffic from different datasets and DDoS attack data.

(IPS). It is designed to scale across a range of CPU cores and leverage DPDK for acceleration and bypassing the kernel [14]. It includes a set of approximately 20 thousand rules to mitigate attacks that target networks. The user can write and specify the Suricata rules to be loaded. However, the parameters will always depend on hard-coded thresholds that have to be set. On the other hand, the proposed system avoids hard-coded thresholds by dynamically adapting to network conditions. The experiment involves implementing seven DDoS attack mitigations through specified Suricata rules. To ensure a fair comparison, Suricata's wide suite of predefined rules is disabled, and only configured rules are loaded. For a fair comparison, DPDK is activated when running Suricata. Fig. 7 shows an example of an attack deployed using a Suricata rule. The first block of code is a rule that detects a SYN packet by monitoring TCP packets with the SYN flag set. The second block of code is a threshold and filter configuration. It creates a filter that restricts the number of received SYN packets sent to the same source IP to a maximum of one million per second, dropping any packets exceeding this threshold.

The results in Fig. 8 (a) show that the packet loss decreases on average from 83% to 31% as we increase the number of CPU cores from one to eight.

```

alert tcp any any -> any any (msg:"SYN_packet"; flags:S;
noalert; sid:123;)

rate_filter gen_id 1, sig_id 123, track by_dst, count
1000000, seconds 1, new_action drop, timeout 30

```

Fig. 7. Suricata rule and configuration to mitigate a SYN flood attack.

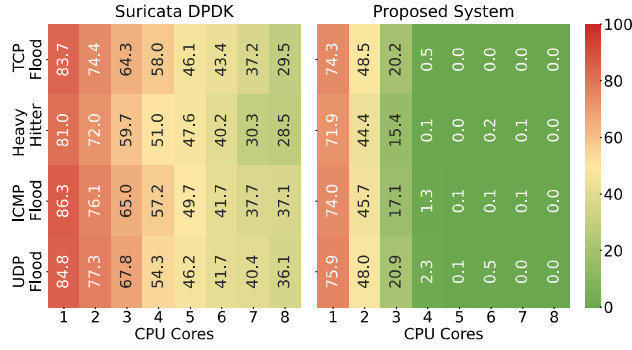


Fig. 8. Packet loss in Suricata-DPDK (a) and the proposed system (b) while mitigating DDoS attacks. Packet loss decreases with more CPU cores. Four Cores are sufficient to eliminate performance penalty.

To evaluate the performance of our system, we measured the packet loss in the processed background traffic while the network was under various DDoS attacks, and the P4-DPDK pipeline was detecting and mitigating the attacks. The measurements shown in Fig. 8 (b) indicate that using more CPU cores, which also involves running more P4-DPDK pipelines in parallel, significantly reduces packet loss. With four or more CPU cores, the packet loss is nearly eliminated.

VI. CONCLUSION AND FUTURE WORK

This work introduces an approach to DDoS detection and mitigation that combines the adaptability of the P4 language with the performance benefits of DPDK. By leveraging a lightweight linear prediction model, the system dynamically computes rate-limiting thresholds in real-time, achieving high detection accuracy without reliance on hard-coded parameters. Evaluation results highlight the system's capability to process traffic at nearly 100 Gbps. The proposed system adapts to varying traffic patterns using real datasets. SYN flood attacks from a dataset are correctly detected among the legitimate real traffic. P4-DPDK outperforms Suricata-DPDK in terms of packet retention during attack scenarios. The system's performance scales up with added CPU cores, reducing packet loss and maintaining performance. Future work will focus on extending detection capabilities and implementing the system entirely in the SmartNIC pipeline.

ACKNOWLEDGMENT

This work was supported by U.S. National Science Foundation, under awards 2346726 and 2403360.

REFERENCES

- [1] D. Warburton, "2024 DDoS attack trends." [Online]. Available: <https://tinyurl.com/35samzj>, 2024. Accessed: Apr. 1, 2025.
- [2] S. Mansfield-Devine, "The evolution of DDoS," *Computer Fraud & Security*, vol. 2014, no. 10, pp. 15–20, 2014.
- [3] T. E. Ali, Y.-W. Chong, and S. Manickam, "Comparison of ML/DL approaches for detecting DDoS attacks in SDN," *Applied Sciences*, vol. 13, no. 5, p. 3033, 2023.
- [4] E. F. Kfoury, S. Choueiri, A. Mazloun, A. AlSabe, J. Gomez, and J. Crichigno, "A comprehensive survey on SmartNICs: Architectures, development models, applications, and research directions," *IEEE Access*, 2024.

- [5] E. F. Kfoury, J. Crichigno, and E. Bou-Harb, "An exhaustive survey on P4 programmable data plane switches: taxonomy, applications, challenges, and future trends," *IEEE Access*, vol. 9, pp. 87094–87155, 2021.
- [6] samiachoueiri Github, "P4-DPDK-Security-LPM." [Online]. Available: https://github.com/samiachoueiri/P4-DPDK_SEC_LPM.git.
- [7] A. Mazloun, A. AlSabe, E. Kfoury, and J. Crichigno, "Security applications in P4: Implementation and lessons learned," *Computer Networks*, vol. 257, 2025.
- [8] L. Tang, Q. Huang, and P. P. Lee, "A fast and compact invertible sketch for network-wide heavy flow detection," *IEEE/ACM Transactions on Networking*, 2020.
- [9] M. Zhang, G. Li, S. Wang, and C. Liu, "Poseidon: mitigating volumetric DDoS attacks with programmable switches," in *The 27th NDSS*, 2020.
- [10] Z. Liu, H. Namkung, G. Nikolaidis, and J. Lee, "Jaqen: A {High-Performance}{Switch-Native} approach for detecting and mitigating volumetric {DDoS} attacks with programmable switches," in *30th USENIX Security Symposium*, pp. 3829–3846, 2021.
- [11] L. Yang, B. Ng, and W. K. Seah, "Heavy hitter detection and identification in software defined networking," in *2016 25th International Conference on Computer Communication and Networks (ICCCN)*, pp. 1–10, IEEE, 2016.
- [12] R. B. Basat, G. Einziger, R. Friedman, M. C. Luizelli, and E. Waisbard, "Volumetric hierarchical heavy hitters," in *2018 IEEE 26th International Symposium on MASCOTS*, pp. 381–392, IEEE, 2018.
- [13] S. Scherrer, C.-Y. Wu, Y.-H. Chiang, and B. Rothenberger, "LOFT: an efficient and scalable algorithm for detecting overuse flows," in *2021 40th International SRDS*, IEEE, 2021.
- [14] Suricata OISF, "Using Capture Hardware - DPDK." [Online]. Available: <https://tinyurl.com/5t43ftv3>. Accessed: Apr. 1, 2025.
- [15] P. Bosshart, D. Daly, G. Gibb, M. Izzard, and N. McKeown, "P4: Programming Protocol-independent Packet Processors," *ACM SIGCOMM Computer Communication Review*, 2014.
- [16] H. Zhu, *Data Plane Development Kit (DPDK): A Software Optimization Guide to the User Space-Based Network Applications*. CRC Press, 2020.
- [17] P4lang, "p4c." [Online]. Available: <https://tinyurl.com/4jfw9bd>. Accessed: Apr. 1, 2025.
- [18] P4lang, "DPDK backend." [Online]. Available: <https://tinyurl.com/cw29ubxa>. Accessed: Apr. 1, 2025.
- [19] DPDK, "DPDK Pipeline Application." [Online]. Available: <https://tinyurl.com/227m8n85>. Accessed: Apr. 1, 2025.
- [20] S. Elzalde, A. AlSabe, A. Mazloun, S. Choueiri, E. Kfoury, J. Gomez, and J. Crichigno, "A survey on security applications with SmartNICs: Taxonomy, implementations, challenges, and future trends," *Journal of Network and Computer Applications*, vol. 242, 2025.
- [21] A. Mazloun, A. AlSabe, E. Kfoury, and J. Crichigno, "Domain name security inspection at line rate: TLS SNI extraction in the data plane using P4 and DPDK," in *IEEE International Conference on Communications (ICC)*, 2025.
- [22] A. AlSabe, K. Friday, E. Kfoury, J. Crichigno, and E. Bou-Harb, "On DGA detection and classification using P4 programmable switches," *Computers & Security*, vol. 145, 2024.
- [23] A. Hikmah, "Understanding Supervised Learning: The Basics of Linear Regression." [Online]. Available: <https://tinyurl.com/yc7bzfww>. Accessed: Apr. 1, 2025.
- [24] A. O'Kinneide, D. Dorran, and M. Gainza, "Linear prediction: The problem, its solution and application to speech," 2008.
- [25] S. Choueiri, A. Mazloun, E. F. Kfoury, and J. Crichigno, "Scalable heavy hitter detection: A DPDK-based software approach with P4 integration," in *GLOBECOM 2024*, 2024.
- [26] I. Baldin, A. Nikolich, J. Griffioen, I. I. S. Monga, and K.-C. Wang, "FABRIC: A national-scale programmable experimental network infrastructure," *IEEE Internet Computing*, 2019.
- [27] Pktgen, "Pktgen-DPDK." [Online]. Available: <https://tinyurl.com/2kcyajcz>. Accessed: Apr. 1, 2025.
- [28] MAWI Working Group, "Packet Traces from WIDE Backbone." [Online]. Available: <https://mawi.wide.ad.jp/mawi/>. Accessed: Apr. 1, 2025.
- [29] CAIDA, "CAIDA Data Server: Index of datasets." [Online]. Available: <https://data.caida.org/datasets/>. Accessed: Apr. 1, 2025.
- [30] S. Kumar and S. Gupta, "SDN TCP-SYN Dataset: A dataset for TCP-SYN flood DDoS attack detection in software-defined networks," *Data in Brief*, 2025.
- [31] Suricata OISF, "Suricata." [Online]. Available: <https://github.com/OISF/suricata>. Accessed: Apr. 1, 2025.